

Step 3 — Multiple MCP Servers at Once, With Tool Namespacing — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume two small servers, `notes_server.py` (tools `search_notes`, `read_note`) and `filesystem_server.py` (tools `list_files`, `read_file`) — each a `MCPServer` server in the same shape as Step 1's `example_server.py`, and this config:

```
# servers_config.py
SERVERS = [
    {"name": "notes", "command": "python", "args": ["notes_server.py"]},
    {"name": "filesystem", "command": "python", "args": ["filesystem_server.py"]}
]
```

Basic Version

Approach 1 — the direct way

```
# server_pool.py
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from tool_bridge import call_mcp_tool

class MCPServerPool:
    def __init__(self):
        self._sessions = {}
        self._routing = {}
        self._stack = AsyncExitStack()

    async def connect_all(self, servers_config):
        openai_tools = []
        for server in servers_config:
            params = StdioServerParameters(command=server["command"], args=server["args"])
            read, write = await self._stack.enter_async_context(stdio_client(params))
            session = await self._stack.enter_async_context(ClientSession(read, write))
            openai_tools.extend(session.list_tools())
```

```

        await session.initialize()
        self._sessions[server["name"]] = session

        tools_result = await session.list_tools()
        for tool in tools_result.tools:
            namespaced_name = server["name"] + "__" + tool.name
            self._routing[namespaced_name] = (server["name"], tool.name)
            openai_tools.append({
                "type": "function",
                "function": {
                    "name": namespaced_name,
                    "description": tool.description or "",
                    "parameters": tool.inputSchema,
                },
            })
        return openai_tools

    async def call_tool(self, namespaced_name, arguments):
        server_name, original_name = self._routing[namespaced_name]
        session = self._sessions[server_name]
        return await call_mcp_tool(session, original_name, arguments)

    async def close(self):
        await self._stack.aclose()

```

Expected output, calling `pool.connect_all(SERVERS)`: an `openai_tools` list where every entry's `name` looks like `notes__search_notes`, `notes__read_note`, `filesystem__list_files`, `filesystem__read_file` — four tools, from two servers, in one flat list.

This works correctly. There's no check for a duplicate `server["name"]` in the config, and `call_tool` will raise a plain `KeyError` if the model somehow sends back a name that was never registered — both fine for a first working version, both fixed below.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, and `agent.py` updated to call through the pool

```
# server_pool.py (unchanged from Basic, plus type hints)
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from tool_bridge import call_mcp_tool

NAMESPACE_SEP = "__"

class MCPServerPool:
    def __init__(self) -> None:
        self._sessions: dict[str, ClientSession] = {}
        self._routing: dict[str, tuple[str, str]] = {}
        self._stack = AsyncExitStack()

    async def connect_all(self, servers_config: list[dict]) -> list[dict]:
        openai_tools: list[dict] = []
        for server in servers_config:
            params = StdioServerParameters(command=server["command"], args=server
            read, write = await self._stack.enter_async_context(stdio_client(para
            session = await self._stack.enter_async_context(ClientSession(read, w
            await session.initialize()
            self._sessions[server["name"]] = session

            tools_result = await session.list_tools()
            for tool in tools_result.tools:
                namespaced_name = f"{server['name']}{NAMESPACE_SEP}{tool.name}"
                self._routing[namespaced_name] = (server["name"], tool.name)
                openai_tools.append({
                    "type": "function",
                    "function": {
                        "name": namespaced_name,
                        "description": tool.description or "",
                        "parameters": tool.inputSchema,
                    },
                })
        return openai_tools
```

```

async def call_tool(self, namespaced_name: str, arguments: dict) -> str:
    server_name, original_name = self._routing[namespaced_name]
    session = self._sessions[server_name]
    return await call_mcp_tool(session, original_name, arguments)

async def close(self) -> None:
    await self._stack.aclose()

```

```

# agent.py -- only the tool-calling round changes from Step 2
...
for tool_call in message.tool_calls:
    name = tool_call.function.name          # now a namespaced name, e.g. "notes_
    arguments = json.loads(tool_call.function.arguments)
    log.append({"step": step, "type": "tool_call", "tool": name, "arguments": arg

    result_text = await pool.call_tool(name, arguments) # was call_mcp_tool(ses
    log.append({"step": step, "type": "observation", "tool": name, "result": resu

    messages.append({"role": "tool", "tool_call_id": tool_call.id, "content": res
...

```

Expected output: identical run shape to Step 2, except the loop's `run_agent` signature now takes `pool: MCPServerPool` instead of `session: ClientSession`, and each log entry's `tool` field shows exactly which server the call belonged to (the `notes__` or `filesystem__` prefix), not just a bare tool name.

Difference from Basic: full type hints, a named `NAMESPACE_SEP` constant instead of a magic string repeated in two places, and `agent.py` shown updated to call through the pool — Step 2's loop logic doesn't change at all, only *what* it calls to actually run a tool.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — guards against duplicate server names and an unknown tool call

```
# server_pool.py
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from tool_bridge import call_mcp_tool

NAMESPACE_SEP = "__"

class UnknownToolError(Exception):
    pass

class MCPServerPool:
    def __init__(self) -> None:
        self._sessions: dict[str, ClientSession] = {}
        self._routing: dict[str, tuple[str, str]] = {}
        self._stack = AsyncExitStack()

    async def connect_all(self, servers_config: list[dict]) -> list[dict]:
        seen_names = set()
        for server in servers_config:
            if server["name"] in seen_names:
                raise ValueError(f"Duplicate server name in config: '{server['name']}'")
            seen_names.add(server["name"])

        openai_tools: list[dict] = []
        for server in servers_config:
            params = StdioServerParameters(command=server["command"], args=server["args"],
            read, write = await self._stack.enter_async_context(stdio_client(params))
            session = await self._stack.enter_async_context(ClientSession(read, write))
            await session.initialize()
            self._sessions[server["name"]] = session

            tools_result = await session.list_tools()
            for tool in tools_result.tools:
                namespaced_name = f"{server['name']}{NAMESPACE_SEP}{tool.name}"
```

```

        self._routing[namespaced_name] = (server["name"], tool.name)
    openai_tools.append({
        "type": "function",
        "function": {
            "name": namespaced_name,
            "description": tool.description or "",
            "parameters": tool.inputSchema,
        },
    })
    return openai_tools

async def call_tool(self, namespaced_name: str, arguments: dict) -> str:
    if namespaced_name not in self._routing:
        raise UnknownToolError(f"'{namespaced_name}' was never discovered from
server_name, original_name = self._routing[namespaced_name]
    session = self._sessions[server_name]
    return await call_mcp_tool(session, original_name, arguments)

async def close(self) -> None:
    await self._stack.aclose()

```

```

# main.py
import asyncio
from servers_config import SERVERS
from server_pool import MCPServerPool
from agent import run_agent

async def main():
    pool = MCPServerPool()
    try:
        openai_tools = await pool.connect_all(SERVERS)
        print(f"Connected. {len(openai_tools)} tools available: {[t['function']]")

        result = await run_agent(
            "Check my notes for anything about the deploy process, and also check
            'if there's a deploy script in the project folder.'",
            pool,
            openai_tools,
        )
        print(result["answer"])
        for entry in result["log"]:
            print(entry)
    finally:
        await pool.close()

```

```
asyncio.run(main())
```

Expected output: the connect line lists all 4 namespaced tools; the log shows at least one `notes__...` call and at least one `filesystem__...` call within the same run, proving the single task actually routed to both servers — the direct test of task 3 from **A Real Example**.

Difference from Intermediate, and between these Advanced pieces: Intermediate's pool works correctly but trusts the config and every incoming tool name blindly. This Approach adds two real guards: a duplicate `server["name"]` in the config fails loudly at `connect_all` time (before any subprocess is even launched), and a namespaced name the model somehow produces that was never actually discovered raises a named `UnknownToolError` instead of a confusing `KeyError` two lines down. The `main.py` shown also demonstrates the `try / finally` cleanup discipline — `pool.close()` runs even if the agent raises partway through, so a crash never leaves orphaned server subprocesses running.

Which one should you actually write? Advanced Approach 1. The duplicate-name guard and the `UnknownToolError` are both cheap to add and turn a confusing failure (a `KeyError` on some internal dict, or two servers silently overwriting each other's tools) into a clear, named one — exactly Doc02's error-handling instinct, applied here. The `try / finally` around `pool.close()` in `main.py` is not optional once you're running real subprocesses; skipping it is how you end up with orphaned server processes still running in the background after your script has already exited.